

TP2 - Rapport d'audit initial UrbanHub

1. Contexte et perimetre

Cet audit initial couvre le depot `xtreme-programming` sur la branche `master`, avec un focus sur les deux services Python exposes:

- `iot-service` (FastAPI)
- `alert-service` (FastAPI)

Objectifs:

- identifier rapidement les vulnerabilites dans le code applicatif;
- identifier les vulnerabilites dans les dependances;
- verifier l'absence d'exposition evidente de secrets dans le depot Git;
- produire une priorisation exploitable pour un RSSI.

Perimetre scanne:

- code source Python sous `iot-service/src` et `alert-service/src`;
- dependances resolues via `uv.lock`;
- historique Git et fichiers suivis pour la recherche de secrets.

2. Justification du combo d'outils

Choix retenu

- SAST: `Bandit`
- SCA: `Trivy fs`
- Secret scan: `GitLeaks`

Justification par matrice de decision

| Critere | Bandit | Trivy fs | Gitleaks | |---|---|---|---| | Adequation a la stack | Excellent pour Python | Excellent pour dependances Python lockees | Standard tres repandu, simple a lancer | | Facilité dexecution locale | Tres bonne | Bonne | Tres bonne | | Sortie exploitable | JSON detaille avec CWE et severite | JSON riche avec CVE, CVSS et version corrgee | SARIF / JSON pour preuves | | Rapport signal / bruit | Bon sur petits services Python | Bon si on cible les services et ignore les environnements | Bon sur le depot Git, plus bruiteux sur `.venv` | | Pertinence pedagogique | Met en evidence les patterns de code risqués | Couvre les libs tierces, angle indispensable | Repond au risque d'exposition accidentelle de secrets | | Integration CI future | Facile | Tres facile | Tres facile |

Pourquoi ce trio est adapte a UrbanHub

`Bandit` est le meilleur choix ici car UrbanHub est en Python/FastAPI et l'objectif est un premier audit pragmatique. Il detecte vite les usages sensibles (`urlopen`, gestion trop large des exceptions, generateur aleatoire non crypto, etc.) avec un cout de mise en oeuvre tres faible.

`Trivy fs` complete l'analyse cote supply chain. Le projet depend de `FastAPI`, donc indirectement de `Starlette`; l'audit a justement remonte deux CVE corrgees sur `starlette@1.0.0`. Ce type de faiblesse ne serait pas visible via un SAST seul.

`GitLeaks` est pertinent pour verifier un risque different: l'exposition de secrets dans le depot et son historique. Le scan du depot Git ne remonte aucun secret confirme. Le scan brut du filesystem remontait uniquement des faux positifs dans des dossiers `.venv`, exclus du rapport executif.

3. Execution effective

Methode recommandee (sortie ordonnee)

Pour eviter un terminal illisible, utiliser le script:

```
powershell -ExecutionPolicy Bypass -File evidence/run-audit.ps1
```

Ce script execute les scans dans l'ordre et genere des logs propres:

- evidence/logs-bandit.txt
- evidence/logs-trivy.txt
- evidence/logs-gitleaks.txt

Ces fichiers sont adaptes aux captures d'ecran du rendu.

Commandes executees

SAST:

```
docker run --rm -v "${PWD}:/src" -w /src python:3.12-slim bash  
-c "pip install -q bandit && bandit -r iot-service/src  
alert-service/src -f json -o evidence/bandit.json"
```

SCA:

```
docker run --rm -v "${PWD}:/src" aquasec/trivy:latest fs --quiet  
--no-progress --severity HIGH,CRITICAL --scanners vuln  
--skip-dirs  
/src/iot-service/.venv,/src/.venv,/src/.git,/src/.idea,/src/site  
--format json -o /src/evidence/trivy-iot.json /src/iot-service  
docker run --rm -v "${PWD}:/src" aquasec/trivy:latest fs --quiet  
--no-progress --severity HIGH,CRITICAL --scanners vuln  
--skip-dirs  
  
/src/alert-service/.venv,/src/.venv,/src/.git,/src/.idea,/src/site --format json -
```

Secrets:

```
docker run --rm -v "${PWD}:/src" -w /src  
zricethezav/gitleaks:latest detect --report-format sarif  
--report-path /src/evidence/gitleaks-git.sarif --no-banner
```

Fichiers de preuve generes

- evidence/bandit.json
- evidence/trivy-iot.json
- evidence/trivy-alert.json
- evidence/gitleaks-git.sarif
- evidence/pip-audit-iot.json
- evidence/pip-audit-alert.json

4. Synthese executive

Niveau de risque global

Risque initial **modere a eleve**.

Les principaux risques identifies ne sont pas des secrets exposes, mais:

- deux vulnerabilites de dependance `Starlette` presentes dans les deux services Python;
- un appel reseau potentiellement dangereux via `urllib.request.urlopen`;

- une gestion d'exception qui masque les erreurs et complique la detection d'incidents.

Chiffres cles

- SAST Bandit: **4 findings**
- SCA Trivy: **4 findings confirmés** (2 CVE, presentes dans 2 services)
- Secrets Git: **0 secret confirme**

5. Top findings priorises

Vue d'ensemble

ID	Outil	Objet	Criticite	Findings
F-01	Trivy	iot-service -> starlette@1.0.0	HIGH / 7.5	
F-02	Trivy	alert-service -> starlette@1.0.0	HIGH / 7.5	
F-03	Trivy	iot-service -> starlette@1.0.0	HIGH / 7.5	
F-04	Trivy	alert-service -> starlette@1.0.0	HIGH / 7.5	
F-05	Bandit B310	hubeau_qualite_client.py:201	MEDIUM	
F-06	Bandit B110	websocket.py:90	LOW	
F-07	Bandit B311	generator.py:51	LOW	
F-08	Bandit B311	orchestrator.py:118	LOW	

Detail des findings

F-01 - Vulnerabilite de dependance dans iot-service

- Outil: Trivy
- Composant: starlette@1.0.0
- Severite: HIGH, CVSS 7.5
- CWE: CWE-918
- Resume: risque de SSRF et de fuite potentielle d'identifiants NTLM via chemin UNC sous Windows
- Exploitabilite estimee: moyenne a elevee si des fichiers statiques sont exposes

F-02 - Vulnerabilite de dependance dans alert-service

- Outil: Trivy
- Composant: starlette@1.0.0
- Severite: HIGH, CVSS 7.5
- CWE: CWE-918
- Resume: meme vulnerabilite que F-01, avec impact potentiel sur le service d'alertes
- Exploitabilite estimee: moyenne a elevee si la surface HTTP est exposable

F-03 - Risque de denial-of-service dans iot-service

- Outil: Trivy
- Composant: starlette@1.0.0
- Severite: HIGH, CVSS 7.5
- CWE: CWE-770
- Resume: les limites de `request.form()` peuvent etre ignorees pour `application/x-www-form-urlencoded`
- Exploitabilite estimee: elevee si des endpoints acceptent des formulaires

F-04 - Risque de denial-of-service dans alert-service

- Outil: Trivy
- Composant: starlette@1.0.0
- Severite: HIGH, CVSS 7.5
- CWE: CWE-770
- Resume: meme vulnerabilite que F-03 sur le service d'alertes
- Exploitabilite estimee: elevee si des endpoints acceptent des formulaires

F-05 - Appel HTTP a encadrer

- Outil: Bandit (B310)
- Fichier: `iot-service/src/iot_service/hubeau_qualite_client.py:201`
- Severite: MEDIUM
- CWE: CWE-22
- Resume: usage de `urllib.request.urlopen()` a auditer pour les schemas autorises
- Exploitabilite estimee: moyenne

F-06 - Exception silencieuse

- Outil: Bandit (B110)
- Fichier: `alert-service/src/alert_service/websocket.py:90`
- Severite: LOW
- CWE: CWE-703
- Resume: `try/except/pass` masque une erreur de fermeture WebSocket
- Exploitabilite estimee: faible a moyenne

F-07 - Generateur pseudo-aleatoire non cryptographique

- Outil: Bandit (B311)
- Fichier: `iot-service/src/iot_service/simulator/generator.py:51`
- Severite: LOW
- CWE: CWE-330
- Resume: `random.Random()` n'est pas adapte a un usage de securite
- Exploitabilite estimee: faible dans le contexte actuel de simulation

F-08 - Alea non cryptographique pour le jitter

- Outil: Bandit (B311)
- Fichier: `iot-service/src/iot_service/simulator/orchestrator.py:118`
- Severite: LOW
- CWE: CWE-330
- Resume: `random.uniform()` n'est pas adapte a un usage de securite
- Exploitabilite estimee: faible dans le contexte actuel de simulation

6. Analyse contextuelle des findings

F-01 / F-02 - Starlette CVE-2026-48818

Les deux services reposent sur FastAPI, qui embarque Starlette. La version verrouillee `starlette@1.0.0` est vulnerable a un scenario SSRF sous Windows avec tentative de resolution de chemin UNC, pouvant provoquer une fuite NTLM.

Dans votre contexte de TP sous Windows, cette remontee est a prendre au serieux meme si l'exploitabilite exacte depend de l'usage de `StaticFiles`.

F-03 / F-04 - Starlette CVE-2026-54283

La version `starlette@1.0.0` est egalement vulnerable a un contournement des limites de parsing pour les corps `application/x-www-form-urlencoded`, avec impact de type denial-of-service.

Cette faille est tres pertinente pour des APIs exposees publiquement.

F-05 - Appel externe via `urlopen`

Le point ci-dessous merite revue:

```
f"?code_station={station_code}"
```

```

        f"&code_parametre={parameter_code}"
        f"&size=20"
        f"&sort=desc"
    )
    try:
        with urllib.request.urlopen(url,
            timeout=self._timeout) as resp:
            payload = json.loads(resp.read())

```

Le risque n'est pas un SSRF prouvé en l'état, car l'URL semble construite depuis une base constante. En revanche, l'outil signale correctement qu'un appel `urlopen` doit être justifié, encadré et si possible remplacé par un client HTTP mieux contrôlé.

F-06 - Exception masquée

```

    except Exception:
        logger.warning("WebSocket send failed, closing
connection")
        try:
            await ws.close()
        except Exception:
            pass
        return False

```

Le `pass` empêche toute trace précise d'un problème de fermeture, ce qui dégrade la détectabilité et l'investigation d'incident.

F-07 / F-08 - Usage de random

```

def __init__(self, sensor: SensorProfile, seed: int | None =
None):
    self.sensor = sensor
    self.rng = random.Random(seed)
    # Current state for random walk

    wait_time = self._config.interval_seconds +
random.uniform(
    -self._config.jitter_seconds,
    self._config.jitter_seconds,
)

```

Ces findings sont faibles car ce code sert à la simulation et non à un mécanisme de sécurité. Ils doivent être gardés dans le rapport, mais clairement de-priorisés.

7. Plan de remédiations

ID	Action proposée	Type	Propriétaire	SLA proposée	--- --- --- --- ---	F-01	Mettre à jour FastAPI/Starlette pour obtenir au minimum <code>starlette>=1.1.0</code> , puis retester les routes statiques Upgrade Dev backend 7 jours
F-02	Même action que F-01 sur <code>alert-service</code>	Upgrade	Dev backend	7 jours		F-03	Mettre à jour FastAPI/Starlette vers une version embarquant <code>starlette>=1.3.1</code> ; ajouter en plus des limites de payload côté reverse proxy/API Upgrade + mitigation Dev backend + Ops 7 jours
F-04	Même action que F-03 sur <code>alert-service</code>	Upgrade + mitigation	Dev backend + Ops	7 jours		F-05	Remplacer <code>urllib.request.urlopen</code> par <code>httpx</code> ou verrouiller strictement l'host, le schema et le timeout; documenter le flux sortant autorisé Patch / mitigation Dev backend IoT 14 jours
F-06	Remplacer <code>pass</code> par une journalisation explicite de l'erreur de fermeture ou une exception contrôlée	Patch	Dev backend	Alert 14 jours		F-07	Accepter le risque si le module reste limité à la simulation, sinon remplacer par <code>secrets</code> si réutilisation sécuritaire future Acceptation / patch Dev backend IoT 30 jours
F-08	Même action que F-07	Acceptation / patch	Dev backend IoT	30 jours			

8. Resultat du secret scan

Le scan `Git leaks` sur le depot `Git` ne remonte **aucun secret confirme**.

Un premier scan brut du filesystem remontait 7 resultats, mais ils provenaient tous de dossiers `.venv` locaux et correspondaient a des faux positifs. Ils ne doivent pas etre inclus dans le rapport executif.

9. Conclusion

L'audit initial UrbanHub ne revele pas de secret expose dans le depot, mais il met en evidence une dette de mise a jour de dependances autour de `Starlette`, ainsi qu'un petit nombre de points de code a corriger ou justifier.

La priorite RSSI doit etre:

- 1 corriger les CVE `Starlette`;
- 2 encadrer l'appel sortant dans `hubeau_qualite_client.py`;
- 3 nettoyer les exceptions silencieuses;
- 4 documenter l'acceptation des findings lies a la simulation.

10. Finalisation du rendu

Actions recommandees avant depot:

- 1 Joindre le rapport PDF `evidence/urbanhub-audit-report.pdf`.
- 2 Joindre le dossier `evidence/` avec les sorties JSON et SARIF.
- 3 Ajouter 3 captures d'ecran lisibles:
 - `evidence/logs-bandit.txt`
 - `evidence/logs-trivy.txt`
 - `evidence/logs-gitleaks.txt`
- 4 Si l'enseignant exige strictement un "top 10", preciser qu'il n'y a que **8 findings confirmes** sur le perimetre scanne.